

AN APPLICATION TO THE TRAVELLING SALESMAN PROBLEM

Aastha Kotecha, Alexander Turner, Dinesh Ramesh, Jaden Noronha, Priscilla Yan
CO 370 – Deterministic OR Models
University of Waterloo
Fall 2023

Table of Contents

Introduction	2
<i>Objective Problem and its value in the real world.....</i>	<i>2</i>
<i>Background on TSP</i>	<i>2</i>
Our Problem	2
<i>Choosing parameter values and constants</i>	<i>2</i>
<i>Choosing our data set for 50 countries.....</i>	<i>3</i>
Formulation.....	4
Methodology.....	5
<i>Improvement 1: Distance from Base Country to any other Country</i>	<i>6</i>
<i>Improvement 2: Distance Between any Two Countries.....</i>	<i>6</i>
<i>Improvement 3: The Triangle Method</i>	<i>6</i>
Results.....	7
Next Steps	7
Other Applications.....	8
Conclusion	8
Appendix A: List of Airports.....	10
Appendix B: 5 Most Efficient Routes from the Example in Methodology	11
References.....	12

Introduction

Objective Problem and its value in the real world

This research project addresses the optimization of travel routes within the tourism sector, specifically focusing on visiting the top destinations globally. Practically, it is not possible for tourists to visit all the top destinations at a go, so in this project we try to find the best places a tourist can visit considering the amount of vacation time they have in hand. With the ever-growing interest in international travel, the application of Travelling Salesman Problem (TSP) in this context aims to streamline route planning, enhancing the efficiency and overall experience for both travelers and tour operators. We will use different algorithms to find the number of countries that can be visited in the time frame provided by the tourist, and we will be using TSP to solve for the optimal route for the countries that can be visited.

Background on TSP

The Traveling Salesman Problem, or TSP for short, is one of the most intensively studied problems in computational mathematics. In the realm of operational research, the Classical Traveling Salesman Problem (TSP) is approached with the aim of determining an optimal route for visiting a predefined set of places precisely once, concluding with a return to the initial destination^[5]. The primary objective lies in strategically determining the most efficient travel routes between locations, based on the associated costs or distances. The formulation portion of TSP can be found in the “[Formulation](#)” section of our project.

Our Problem

Choosing parameter values and constants

To define the parameters for our problem, we chose to work with a dataset encompassing $n = 50$ countries. This choice offers a diverse array of options while remaining computationally manageable. Recognizing that passenger priorities center around the time taken for the tour, we need to convert the distance travelled into time spent traveling. In order to make this conversion we assumed that the airplanes would travel at a constant speed of $s = 850 \text{ km/hr}$ ^[2], which is the average speed of an airplane^[2]. We need to make the above assumption as we are unable to accurately find the time required to fly from one place to the other as we cannot be certain on the weather and the specific plane being used.

To implement the choice of the x countries, we used a systematic approach. One country is identified as the tourist's country of origin or their home base, and it is paired with a list of x more countries. This method is designed to create a tour that takes the tourist both into and out of the home base country. For our test case, we have chosen x to be 5, a value that is challenging

and requires the use of tools learned in CO 370 but also more realistic than larger values of x to be done in real life.

By employing these parameters, we aim to set up a framework that not only presents a diverse set of country options but also addresses passenger concerns regarding travel time under the assumption that the speed traveled is constant. Given these considerations in mind, our approach is tailored to consider numerous factors influencing the overall tour experience.

Choosing our data set for 50 countries

Since countries are large areas rather than discrete points, it is difficult for us to formulate our problem using the countries as the places in the TSP. Therefore, rather than thinking of countries as the locations we are visiting, we will think of the airports within the countries as the points we are visiting, allowing the airports to represent the respective countries.

To choose our 50 airports (thus our 50 countries)^{[3][4]}, what we wanted to do is make sure that we did not visit any country twice, and that the places we were visiting are places in the countries people would want to visit. First to narrow down the countries we decided to look at the countries that had at least 2 large airports under the justification that a country having more large airports means that the country would likely have more tourism (this is not inherently true and there were some examples of places having more than 1 airport not actually having as much tourism based on the TripAdvisor^[1] reviews, despite this we choose to stick with this methodology because the TripAdvisor^[1] reviews were not always consistent within the country which would lead to us needing to make much more arbitrary choices in deciding countries).

This narrowed down the countries to 53, so we removed 3 of the countries that did not fit as well with our project from the list (this was mostly arbitrary as we wanted 50 countries for our list but ultimately should not matter too much) to get a list of 50 countries, however since all of these 50 countries had at least 2 airports we needed to choose which airports we wanted to use for our problem, and since we did not want to revisit the same country, this meant choosing 1 airport from each country.

To decide the airport we wanted to choose, we decided the best thing to do would be to choose an airport that was closest to a popular tourist destination in the given country. To stay consistent with our choices we decided to use TripAdvisor^[1] to see the best place to visit in the country and then either used whatever airport was in the same city, or an airport that seemed to be close to that location from a viewing of Google Maps to decide our airport.

Formulation

There are certain merits in addressing TSP as a Linear Program (LP). The LP formulation offers a very clear formulation making it easy to understand and analyze. Since LP formulation is a standardized format for expressing various optimization problems, there are well established framework with efficient algorithms for solving large scale linear programs, like our problem. So, we will now formulate our problem as a LP and introduce variables.

The formulation of the TSP problem follows as described:

- Define d_{ij} to be the distance between airport i and airport j . We calculate this distance using the coordinates of the airports.
- Define X_{ij} binary variable deciding whether the tour travels between airports i and j .
- The IP formulation is,

$$\begin{aligned} \min \quad & \sum_{1 \leq i < j \leq 50} d_{ij} X_{ij} \\ \text{s.t.} \quad & \sum_{j \neq i} X_{ij} = 2 \quad \forall i = 1, \dots, 50 \\ & X_{ij} = X_{ji} \quad \forall 1 \leq i < j \leq 50 \\ & X_{ij} \in \{0, 1\} \quad \forall i \neq j \end{aligned}$$

- We can also formulate it as an LP relaxation by changing $X_{ij} \in \{0, 1\}$ to $0 \leq X_{ij} \leq 1$.
- The first constraint makes sure that if the tour enters an airport, then the tour should exit an airport. This essentially means that our tour would starts at an airport and ends at the same airport. The second constraint represents that travelling from airport i to airport j is the same as travelling from airport j to airport i . The last constraint forces the variable to be binary.
- Now, formulating our problem in this way poses a new challenge. There is a possibility that a tour satisfies our LP formulation but does not satisfy our actual problem, for example, if the tour contains two separate cycles. To avoid that we will have to introduce a new set of constraints called the subtour elimination constraint. These set of constraints enables the LP to give us a valid tour to our main problem. This part of the formulation makes this problem as an NP-complete.

- For each set of $S \subseteq \{1, \dots, 50\}$ where $S \neq \emptyset$ and $S \neq \{1, \dots, 50\}$, the corresponding subtour elimination constraint is,

$$\sum (X_{ij} : i \in S, j \in \{1, \dots, 50\} \setminus S) \geq 2$$

- This method of introducing new constraints to get an optimal solution to our liking is called Cutting planes method.

Methodology

To obtain information about the position of the airports in each of our top countries, we utilized data from Esri Germany, containing geographical information on all the airports in the world, including their latitude and longitude. To enrich the data provided by Esri, we also utilized a dataset with ISO-to-country conversions^[3], so we knew which airport belonged to each country. These datasets were then filtered and cleaned in accordance with the selection criteria mentioned in the “[Our Problem](#)” section, utilizing SQL.

Once our airport data on the top 50 countries was obtained, it was imported into python using the Pandas library. Each airport was given an ID from 1 to 50 (refer to [Appendix A](#) for this list), and a dictionary named ‘*coords*’ was created containing the coordinates for all the airports. Using this dictionary, another dictionary named ‘*dist*’ was created, containing the distance between any two airports in kilometers. Since our data source utilized latitude and longitude for its coordinates, the Geopy library was used to calculate this distance in kilometers, using the distance.distance method. To ensure that every possible combination of airports had their distance calculated, the itertools library was used, more specifically its ‘combinations’ function.

To determine the possible tours that meet our requirements, we follow the following algorithm:

- Generate list of permutations of airports that contain the base airport (b) and $M \geq 2$ other countries.
- If any permutation contains a country $c \neq b$ such that the distance between b and c is greater than or equal to the maximum distance allowed divided by two, remove that permutation (Improvement [1](#)).
- If any permutation contains two countries x, y such that the distance between x and y is greater than or equal to the maximum distance allowed divided by two, remove that permutation (Improvement [2](#)).
- If any permutation contains a subset of 3 countries such that the tour containing these countries has an optimal value greater than the maximum distance allowed, remove that permutation. (Improvement [3](#)).
- Run the TSP algorithm for each of the remaining tours. If that tour has an optimal value less than or equal to our maximum distance allowed, add that permutation, the objective value, and the tour path to a dictionary.

For testing purposes, we assumed that we want our tour to start at *Lester B. Pearson International Airport*(ID:50) and we want to visit 5 other countries on our tour, travelling a maximum of 35 hr, which we then convert to distance using the average speed of 850 km/hr, to get 30,000 km approximately. This means that we are calculating tours of 6 countries out of 50 possible countries, where one country in our tour is fixed. This leaves us with ${}^{49}C_5 = 1906884$ possible tours not accounting for our distance constraint. To determine the possible tours in a feasible amount of time, we must reduce the total number of cases. For this, we propose some improvements in the algorithm of our problem that will make it much more efficient.

Improvement 1: Distance from Base Country to any other Country

Assume we want all our tours to start from an airport A . Then for any other airport B in our tour, the shortest distance between them is a straight line, which is given by $dist_{AB}$. If we consider a tour where we go from A to B , then from B to A , the total distance travelled would be $2 \cdot dist_{AB}$. Since we have a maximum distance of M , this implies that $dist_{AB} \leq \frac{M}{2}$. Since we want our tours to visit at least 3 countries (start from one airport, visit at least 2 others), the optimum distance of the total tour would be greater than $2 \cdot dist_{AB}$ as more countries must be visited.

Thus, if $dist_{AB} \geq \frac{M}{2}$ (A is the base airport and B is any other airport in the combination of airports), the tour will not satisfy our distance requirement. Implementing code to remove these constraints results in 1533939 valid tours left, reducing the amount by 19.56%.

Improvement 2: Distance Between any Two Countries

We can extend the previous idea to any two airports X and Y in a combination of airports. If $dist_{XY} \geq \frac{M}{2}$ for any X and Y in a combination of airports, then the combination is not valid. Implementing code to remove tours that violate this requirement reduces our potential tours to 833779, a 56.28% reduction for our initial number of cases.

Improvement 3: The Triangle Method

Idea: If we are going to 2 or more other countries, then the distance of a cycle for any 3 countries will be smaller than or equal to the optimal path for all $1 + n$ countries.

Proof: We take any three countries from selected countries, make a cycle (aka triangle), the distance to walk that cycle will be smaller than the optimal path for visiting all selected countries. It is because, the longest distance we could find for any cycle would be smaller than or equal to the optimal distance due to triangle inequality (For any triangle the sum of the two shorter side is longer than the longer side) and for three points in a straight line, the sum of the 2 smaller distances between two points is equal to the longest distance between two points. Hence for all cycles of 3 countries within selected countries, the distance for the cycles would be smaller or equals to the optimal distance for all selected countries.

Implementing this method, we reduce our total cases to 399141, a 79.07% reduction from our initial number.

After reducing the number of cases, the TSP algorithm (covered in Tutorial 3) can be applied to each of them. Tours whose optimal values were under the maximum distance were added to a dictionary, alongside its optimal value and the route the tour takes.

Results

With these improvements, we end up with something that is much more reasonable computable than before, and we found that the best route starting from YYZ is $50 \rightarrow 33 \rightarrow 35 \rightarrow 49 \rightarrow 34 \rightarrow 30 \rightarrow 50$ which travels $10877km$. There are a total of 295619 possible tours, among which the 5 shortest tours that meet our criteria can be found in [Appendix B](#). While this is the result we got, there is still much more that can be done with the formulation we used formulation and our methodology.

Next Steps

For possible expansions on this there are two methods we can consider, first we could consider improvements made to how we run the problem, the other method is to change the model in interesting ways to consider factors that might be important to consider.

In our current methodology, we took steps to reduce the number of base cases, based on the distance between any two countries (Improvements [1](#) and [2](#)) and the distance for any cycle of length 3 (Improvement [3](#)). However, we can generalize Improvement 3 to take into consideration any number of countries. Let N be a set representing the base country and any other countries in a possible tour, based on the problem constraints. Then, for any $3 \leq k \leq |N| - 1$, if there is a subset of k countries from N that has an optimal minimal tour with length $\geq \text{max_distance}$, then the objective value for the minimum tour for the countries in N is $> \text{max_distance}$.

Proof: Let S be a subset of $3 \leq k \leq |N| - 1$ countries from N , where the optimal value for the TSP is $T \geq \text{max_distance}$. Then, there are $|N| - k \geq 1$ countries from N not included in S that must be considered. Because all the countries in N are distinct, this means that $S + \{c\} \neq S$, where c is in N/S . Since T represents the optimal tour for all the countries in S , the optimal tour for $S + \{c\}$ will deviate from this path, thus the optimal tour for $S + \{c\}$ is $T' > T \geq \text{max_distance}$. Thus, $S + N/S = N$ has an optimal tour length of $T'' > T \geq \text{max_distance}$ and this implies that $T'' > \text{max_distance}$.

We can calculate all the subtours of length k that violate the above statement and eliminate tours which contain the countries in those subtours as a subset. In this implementation, the process

will stop early if there are no valid tours that meet our problem requirements. The drawback is that Gurobi could potentially be run exponentially more times, albeit most of those runs are with smaller tour sizes.

For changing the model, we would want to consider factors that have not been already considered in our formulation, with the most interesting ones to add being: popularity, staying locations, and take off times. For popularity we could weigh the locations so that instead of needing to visit a certain number of locations, we instead need to visit enough locations so that the total weight is at least a certain value, ensuring that the route favors more popular locations. In terms of staying locations, the airports picked did not factor in how easy it would be to find a location to stay at, so one could instead include airports based on the popularity on places to stay near the airport. The last thing to possibly consider in an expansion of the formulation is the takeoff times and landing times, this is because some of the larger airports on the list might have much longer takeoff times or landing times than some of the smaller ones which could influence the locations we choose.

Other Applications

With the structuring we used to formulate this problem, there are many ways we can adjust our problem to consider interesting scenarios. First, by changing data used for our formulation, we can use the formulation in many ways. The 50 airports that we chose are arbitrary, so we could instead add more airports or remove airports that we do not want to consider. We are also allowed to use any number of airports and any airports, so we could formulate the problem to visit a country multiple times or add constraints to limit our visits to a country but allow different airports within the country to be visited. Now this would pose a new challenge of having too many variables, which can be overcome by using a method called Column generation, where we solve for small number of variables and slowly increase the number until we get optimal solution to our liking. We can even use it on trains, buses or even ports as there are no specific reasons to just use airplanes in our formulation. We can also adjust our formulation by modifying the variable a little, for example instead of time or distance, we can set it to price, or anything relating to the relationship of two airports. We can set also add other constraints to the problem that are similar to the maximum travel time on our subproblem, as an example we could limit the max number of times we fly over a large body of water.

Conclusion

The TSP is one of the most famous problems in optimization, and there are multiple ways to represent and solve it. Our approach integrated column generation and cutting planes techniques and using the formulation of the problem as a Mixed-Integer Programming (MIP) model. The use of Gurobi, which employs the branch-and-bound method for solving the integer problem, played a pivotal role in deriving optimal solutions. Overall, we saw how useful the methods learned in

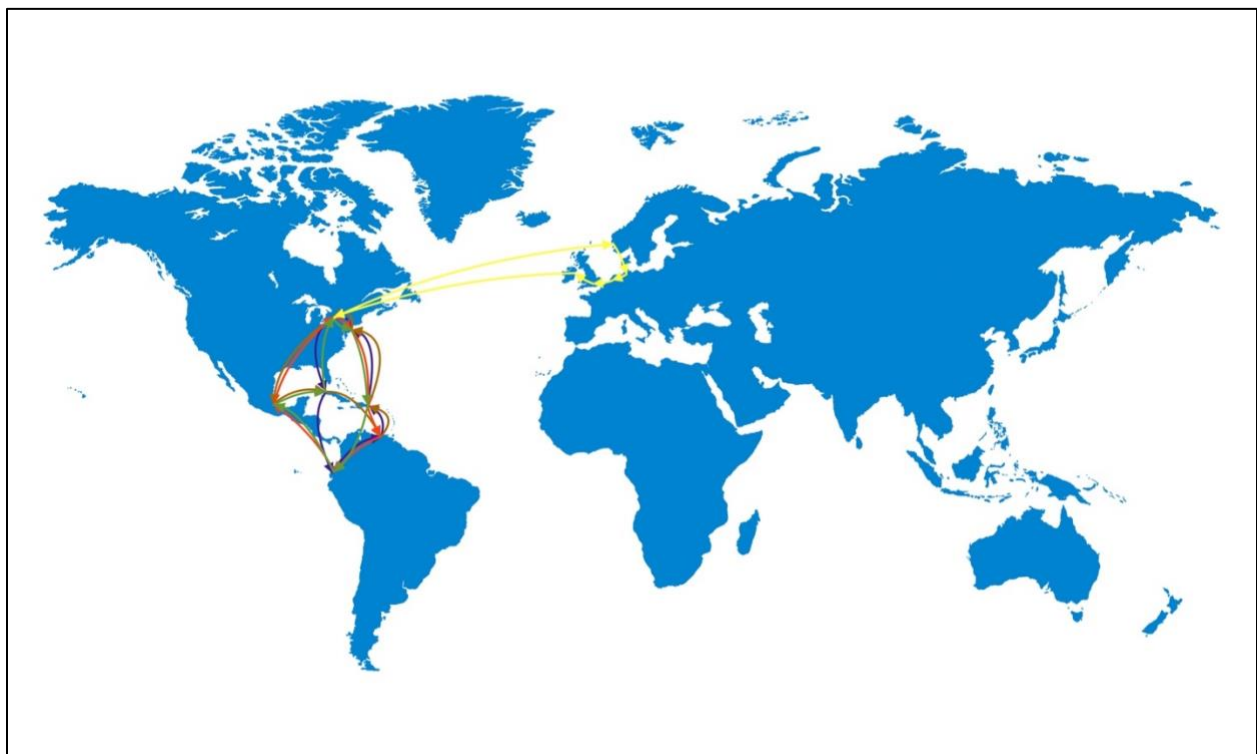
class are in actual applications, and some of the ways we could further explore this problem to tackle broader challenges in the world of optimization.

Appendix A: List of Airports

Index	Airport Name	Country
1	Sydney Kingsford Smith International Airport	Australia
2	Rio Galeão – Tom Jobim International Airport	Brazil
3	Beijing Capital International Airport	China
4	Cairo International Airport	Egypt
5	Paris-Orly Airport	France
6	Hamburg Helmut Schmidt Airport	Germany
7	Athens Eleftherios Venizelos International Airport	Greece
8	Indira Gandhi International Airport	India
9	Ngurah Rai (Bali) International Airport	Indonesia
10	Shiraz Shahid Dastghaib International Airport	Iran (Islamic Republic of)
11	Rome–Fiumicino Leonardo da Vinci International Airport	Italy
12	Osaka International Airport	Japan
13	Almaty International Airport	Kazakhstan
14	Incheon International Airport	Korea, Republic of
15	General Felipe Ángeles International Airport	Mexico
16	Bergen Airport, Flesland	Norway
17	Allama Iqbal International Airport	Pakistan
18	Aeropuerto Internacional Guaraní	Paraguay
19	Ninoy Aquino International Airport	Philippines
20	Kraków John Paul II International Airport	Poland
21	Humberto Delgado Airport (Lisbon Portela Airport)	Portugal
22	Pulkovo Airport	Russian Federation
23	Prince Mohammad Bin Abdulaziz Airport	Saudi Arabia
24	Cape Town International Airport	South Africa
25	Josep Tarradellas Barcelona-El Prat Airport	Spain
26	Suvarnabhumi Airport	Thailand
27	Istanbul Sabiha Gökçen International Airport	Turkey
28	Dubai International Airport	United Arab Emirates
29	London Heathrow Airport	United Kingdom of Great Britain and Northern Ireland
30	La Guardia Airport	United States of America
31	Burgas Airport	Bulgaria
32	Jorge Newbery Airpark	Argentina
33	Juan Gualberto Gomez International Airport	Cuba
34	Punta Cana International Airport	Dominican Republic
35	Mariscal Sucre International Airport	Ecuador
36	Dublin Airport	Ireland
37	Jomo Kenyatta International Airport	Kenya
38	Noi Bai International Airport	Viet Nam

39	Wellington International Airport	New Zealand
40	Murtala Muhammed International Airport	Nigeria
41	Muscat International Airport	Oman
42	Alejandro Velasco Astete International Airport	Peru
43	Amsterdam Airport Schiphol	Netherlands
44	Yangon International Airport	Myanmar
45	Stockholm-Arlanda Airport	Sweden
46	Geneva Cointrin International Airport	Switzerland
47	Bandaranaike International Colombo Airport	Sri Lanka
48	Abeid Amani Karume International Airport	Tanzania, United Republic of
49	General José Antonio Anzoategui International Airport	Venezuela (Bolivarian Republic of)
50	Lester B. Pearson International Airport	Canada

Appendix B: 5 Most Efficient Routes from the Example in Methodology



References

- [1] Tripadvisor. (n.d.). <https://www.tripadvisor.in/>
- [2] *What's the speed of an airplane? - world aviation ato*. World Aviation | Escuela de Pilotos de Referencia. (2023, December 5). <https://worldaviationato.com/en/airplane-speed/>
- [3] *ISO 3166 Countries with Regional Codes*. GitHub. (n.d.). <https://github.com/luke/ISO-3166-Countries-with-Regional-Codes/blob/master/all/all.csv>
- [4] *World airports*. ArcGIS Hub. (n.d.). <https://hub.arcgis.com/datasets/esri-de-content::world-airports/about>
- [5] *Bandara, D. (n.d.). Application to the travelling salesman problem - AIRCCSE*. <https://airccse.com/oraj/papers/4417oraj02.pdf>